

Document number: P0025

Date: 2015-09-18

Revises: N4536

Project: Programming Language C++, Library Evolution Working Group

Reply to: Martin Moene <martin.moene (at) gmail.com>, Niels Dekker <n.dekker (at) xs4all.nl>

An algorithm to "clamp" a value between a pair of boundary values (revision 1)

Changes since N4536

Funtion `clamp_range()` is considered superfluous in view of the Ranges proposal and has been dropped from this proposal. The declaration style of `clamp()` has been made consistent with the one of `min()` and `max()`.

[Introduction](#)

[Motivation](#)

[Impact on the standard](#)

[Comparison to clamp of Boost.Algorithm](#)

[Design decisions](#)

[Proposed wording](#)

[Possible implementation](#)

[Acknowledgements](#)

[References](#)

Introduction

The algorithm proposed here "clamps" a value between a pair of boundary values. The idea and interfaces are inspired by clamp in the Boost.Algorithm library authored by Marshall Clow.

Motivation

It is a common programming task to constrain a value to fall within certain limits. This can be expressed in numerous ways, but it would be good if such an operation can be easily recognized and doesn't appear in many guises just because it can.

So, we'd like to have a concise way to obtain a value that is forced to fall within a range we request, much like we can limit a value to a defined minimum or maximum. For example:

```
auto clamped_value = clamp( value, min_value, max_value );
```

Without a standardized way, people may (need to) define their own version of "clamp" or resort to a less clear solution such as¹:

```
auto clamped_value = std::min( std::max( value, min_value ), max_value );
```

In addition to the boundary values, one can provide a predicate that evaluates if a value is within the boundary.

```
struct rgb{ ... };
```

```
auto clamped_rgb = clamp( rgb_value, rgb_lo, rgb_hi, rgb_compare );
```

Function `clamp()` already exists in C++ libraries such as Boost [\[1\]](#) and Microsoft AMP [\[2\]](#). The Qt Project provides `qBound` [\[3\]](#), and the Python library `scipy/numpy` provides `clip()` [\[4\]](#) for the same purpose.

Impact on the standard

The clamp algorithms can be implemented as a pure library extension. We suggest to add them to sub-clause 25.4 Sorting and related operations of the Algorithms library.

Comparison to clamp of Boost.Algorithm

Like `std::min()` and `std::max()`, `clamp()` requires its arguments to be of the same type, whereas, Boost's clamp accepts arguments of different type.

Design decisions

We chose the name *clamp* as it is expressive and is already being used in other libraries². Another name could be *limit*.

`clamp()` can be regarded as a sibling of `std::min()` and `std::max()`. This makes it desirable to follow their interface using

constexpr, passing parameters by const reference and returning the result by const reference. Passing values by `const` & is desired for types that have a possibly expensive copy constructor such as `cpp_int` of Boost.Multiprecision [5] and `std::seminumeric::integer` from the Proposal for Unbounded-Precision Integer Types [6].

Proposed wording

25.4.X Bounded value

[alg.clamp]

```
template<class T>
constexpr const T& clamp( const T& v, const T& lo, const T& hi );

template<class T, class Compare>
constexpr const T& clamp( const T& v, const T& lo, const T& hi, Compare comp );
```

1 *Requires*: For the first form, type T shall be LessThanComparable (Table 18).

2 *Returns*: The larger value of v and lo if v is smaller than hi, otherwise the smaller value of v and hi.

3 *Remarks*: Returns the first argument when it is equivalent to one of the boundary arguments.

4 *Complexity*: At most two comparisons.

Possible implementation

Clamp a value:

```
template<class T>
constexpr const T& clamp( const T& v, const T& lo, const T& hi )
{
    return clamp( v, lo, hi, less<T>() );
}
```

Clamp a value per predicate:

```
template<class T, class Compare>
constexpr const T& clamp( const T& val, const T& lo, const T& hi, Compare comp )
{
    return assert( !comp(hi, lo) ),
           comp(val, lo) ? lo : comp(hi, val) ? hi : val;
}
```

Acknowledgements

Thanks to Marshall Clow for Boost.Algorithm's clamp which inspired this proposal and to Daniel Krügler and Jonathan Wakely for their help with the proposing process.

References

[1] Marshall Clow. [clamp in the Boost Algorithm Library](#).

Note: the Boost documentation shows `clamp()` using pass by value, whereas the actual code in [boost/algorithm/clamp.hpp](#) uses `const` &. See [ticket 10081](#).

[2] Microsoft. [C++ Accelerated Massive Parallelism library \(AMP\)](#).

[3] Qt Project. [Documentation on qBound](#).

[4] Scipy.org. [Documentation on numpy.clip](#).

[5] John Maddock. [Boost.Multiprecision](#).

[6] Pete Becker. [Proposal for Unbounded-Precision Integer Types \(N4038\)](#).

1. Or even:

```
auto clamped_value = value;
if ( value < min_value ) clamped_value = min_value;
else if ( value > max_value ) clamped_value = max_value;
```

↩

2. As suggested by Jonathan Wakely on mailing list `accu-general` on 18 February 2014.↩